VOLUME 02 / 08

THE RV32I DATAPATH

Decode instructions and implement their visible results

Implement instruction fields, immediates, arithmetic, control flow and byte-addressed memory behavior.

FIG 04 / ADD DATAFLOW

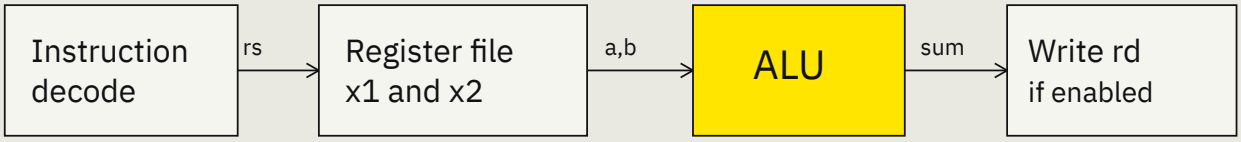


Illustration: values move forward while control selects the permitted update.

A hands-on companion to the RISC-V project. Examples, implementation steps, debugging and completion checks. Personal design edition / October 2026.

What you will build

You will turn instruction words into explicit operations and expected architectural results. Build one instruction family at a time, then compare against the reference.

Before you start

Finish Volume 1. Know binary slices, signed interpretation and clocked state.

Files and outputs

rtl/rv32_core.sv: instruction fields, imm_* wires, opcode case and memory formatting.
tests/run.py: encoders and reference().

Build order

1. Define the supported execution environment.
2. Extract fields and reconstruct immediates.
3. Implement ALU and control-flow cases.
4. Implement load/store lanes and legality.
5. Run small instruction-family programs.

The checked-in core is an assistant-generated reference. Work on a learning branch. The code excerpts explain selected regions and are not complete replacement modules unless explicitly labeled.

The ISA defines visible behavior

An **instruction set architecture**, or ISA, defines the operations software can request and the state it can observe. A microarchitecture chooses how to execute those requests.

RV32I is the RISC-V base integer instruction set with 32-bit integer registers. The integer register file has x0 through x31. x0 always reads zero and discards writes. The **program counter**, or PC, identifies the instruction address.

`add x3, x1, x2` means read x1 and x2, add their 32-bit values, and write x3. `addi x1, x0, 7` uses an immediate constant encoded inside the instruction.

RISC-V separates register arithmetic from memory access. `lw x1, 128(x0)` loads a word from byte address 128. `sw x1, 132(x0)` stores a word at byte address 132.

> **SAME CONTRACT, DIFFERENT EXECUTION**
>
> A single-cycle machine and a five-stage pipeline should produce the same architectural instruction results. Their timing and internal state differ.

Instruction	Inputs	Visible result
addi x1, x0, 7	x0 and immediate 7	x1 = 7
add x3, x1, x2	x1 and x2	x3 = x1 + x2
lw x4, 128(x0)	base 0, offset 128	x4 = memory word
sw x4, 132(x0)	address and x4	memory word = x4

An instruction word names operands and work

RV32I instructions here are 32 bits long. Decode examines fields within that word. The **opcode** selects a broad instruction class. **funct3** and sometimes **funct7** refine the operation.

In an R-type arithmetic instruction, rd names the destination. rs1 and rs2 name source registers. The bit positions stay fixed across several instruction formats, simplifying decode.

> **WORKED EXAMPLE**
>
> `add x3, x1, x2` encodes as `0x002081B3`. Its opcode is `0x33`, funct3 is 0, funct7 is 0, rd is 3, rs1 is 1 and rs2 is 2.

An I-type instruction uses bits 31:20 for a 12-bit immediate. `addi x1, x0, 7` encodes as `0x00700093`. Ordinary arithmetic immediates are sign-extended. Shift immediates have additional encoding rules.

The local tests construct words with Python encoders. A standard assembler is useful later, but is not required to run these regressions.

FIG 03 / R-TYPE FIELD MAP

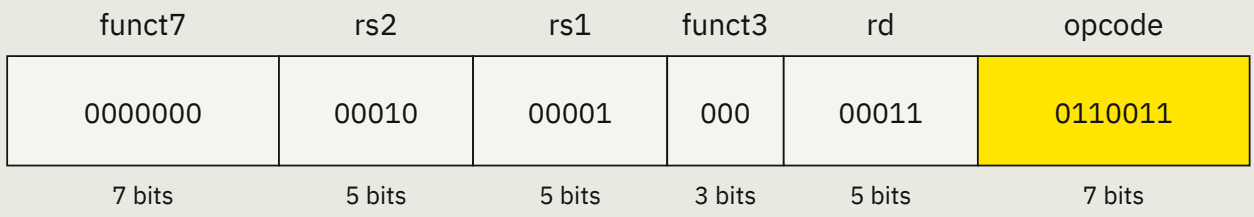


Illustration: field widths sum to 32 bits. Widths use a shared bit scale.

Addresses count bytes

A word contains four bytes. In **little-endian** order, the least significant byte sits at the lowest address. Alignment requires a word address divisible by four and a halfword address divisible by two in this core.

> **WORKED EXAMPLE**
>
> Storing `0x80FF007F` at address 128 writes bytes 7F, 00, FF and 80 at addresses 128, 129, 130 and 131.

`lb x1, 131(x0)` returns `0xFFFFFF80`. `lbu x1, 131(x0)` returns `0x00000080`. The memory byte is identical. The load operation selects sign or zero extension.

The core exposes an aligned data word address plus four byte-write strobes. It uses the low effective-address bits to select a byte or halfword. Misaligned halfword and word accesses produce a local trap.

Instruction and data memories are separate. Both reads must be combinational. There is no ready signal or support for waiting on a slow memory.

Byte address	128	129	130	131
Stored byte	0x7F	0x00	0xFF	0x80
Role in word	bits 7:0	bits 15:8	bits 23:16	bits 31:24

Step 1: define the instruction contract

List each supported operation and its destination, memory effect and next-PC behavior. Keep implementation timing out of this table. ADD writes a sum. SW writes memory. BEQ may select a branch target.

The reference has integer arithmetic, branches/jumps and ordinary byte/halfword/word memory operations. It omits M, compressed instructions, privileged state and interrupts. Unsupported encodings report a local trap and halt.

For an initial learning implementation, begin with ADDI, ADD, LW, SW and EBREAK. Mark that smaller subset explicitly. Expand it before claiming the full reference instruction scope.

Write three examples per family: an ordinary value, a boundary value and a fault or invalid encoding. The standard ISA is the authority for instruction semantics. The local architecture document defines the memory and trap environment.

> **TRY IT BEFORE READING THE ANSWERS**
>
> 2A. Does implementing the ADD opcode alone establish support for every arithmetic operation in RV32I? Explain the missing decode information.

Step 2: extract fields and immediates

The reference decodes the instruction in EX, after ID/EX has captured it. Use the instruction associated with the current operation, not the live fetch word.

```
wire [6:0] op = x_insn[6:0];
wire [2:0] funct3 = x_insn[14:12];
wire [6:0] funct7 = x_insn[31:25];
wire [31:0] imm_i = {{20{x_insn[31]}},x_insn[31:20]};
wire [31:0] imm_s = {{20{x_insn[31]}},
    x_insn[31:25],x_insn[11:7]};
```

Store offsets split across two fields. Branch and jump offsets use another arrangement and include an implicit low zero bit. Reconstruct B and J immediates from the ISA diagram, then compare with `imm_b` and `imm_j`.

Use both positive and negative offsets. A forward branch can pass while a backward loop still fails because the sign or high bit is wrong.

> **TRY IT BEFORE READING THE ANSWERS**
>
> 2B. A branch at PC=12 has byte offset −4. What is the target? Why is the offset not added to PC+4?

Build a minimal single-cycle baseline

The complete source in `docs/textbook/series/labs/single_cycle.sv` implements ADDI, ADD, aligned LW, aligned SW and EBREAK. It is a teaching subset, not a second measured RV32I core.

The PC addresses combinational instruction memory. Register reads, decode, ALU and load data settle before the next edge. A legal instruction updates its register or memory effect at that edge. EBREAK or an unsupported instruction halts.

Start by implementing the PC, reset and register array. Then add opcode/field wires, I/S immediates and ADD/ADDI. Add word-memory ports and alignment checks last. Compare each section with the complete source only after writing your own prediction.

Run `make learning-labs` in the tools container. The standalone bench computes 7+3, stores 10 at address 128, loads it into x4 and checks x0, x3, x4 and memory.

The teaching top has a different, smaller interface and no retirement trace. Do not substitute it for rv32_core in the pipeline testbench. Volume 3 carries the same conceptual datapath into the real five-stage interface.

> **TRY IT BEFORE READING THE ANSWERS**
>
> 2D. Why does an immediately dependent ADD work without forwarding in this single-cycle subset?

Step 3: implement ALU operations

Use funct3 to distinguish operation families and funct7 where required. ADD and SUB share opcode/funct3 and differ in funct7. Logical and arithmetic right shifts also need that distinction.

```
// Selected EX behavior, inside the decode block.
result = a + b;                  // ADD
result = a - b;                  // SUB
result = a << b[4:0];            // SLL
result = $signed(a) >>> b[4:0];   // SRA
result = {31'b0, ($signed(a) < $signed(b))};
```

These lines show alternatives, not a block to append unchanged. The last example produces an SLT result. SLTU uses an unsigned comparison. Register shifts use only the low five amount bits in RV32I.

For immediate operations, choose the sign-extended immediate as the second operand, except for the encoding-specific shift amount. Validate reserved function bits. A permissive decoder can silently execute unsupported instructions.

Run the learning case `decode_add`, then add signed/unsigned comparison and extreme-shift examples in a learning branch.

Step 4: implement branches and links

Use the instruction's saved PC for AUIPC, JAL, branch targets and return addresses. JAL/JALR write PC+4 even though the fetch PC may already be ahead.

```
// JALR conceptual EX values.
result = x_pc + 4;
target = (a + imm_i) & 32'hfffffffe;
taken = 1;
legal = (funct3 == 0);
```

After clearing bit zero, this 32-bit-instruction-only core still rejects a target whose bit one is set. Check alignment only for taken control targets. An untaken branch must not fault because its unused target would be misaligned.

Compare signed branch operands for BLT/BGE and unsigned operands for BLTU/BGEU. Use forwarded values once the pipeline overlaps instructions.

Volume 4 adds the squash and redirect mechanism. Here, the deliverable is correct target and link arithmetic.

Step 5: implement memory lanes

First calculate the effective byte address. Preserve its low bits for lane selection while presenting an aligned word address to memory.

```
assign dmem_addr = {m_result[31:2],2'b0};
assign dmem_wdata = m_store << (8*m_result[1:0]);
wire [31:0] shifted_load =
    dmem_rdata >> (8*m_result[1:0]);
```

For a byte store, shift 0001 by the byte offset. For a halfword, shift 0011 after checking halfword alignment. A word uses 1111 after checking word alignment. Gate all strobes with a valid, permitted store.

LB and LH sign-extend the selected lane. LBU and LHU zero-extend it. LW returns the word. The `word_bytes` teaching program stores 0x80FF007F and verifies both signed and unsigned reads.

Do not treat the testbench's address masking as access-fault support. Generated tests stay in range; real out-of-range memory behavior needs an explicit environment contract.

> **TRY IT BEFORE READING THE ANSWERS**
>
> 2C. Which strobe writes one byte at aligned-word offset 2? What word alignment check is required?

Step 6: test legality and side effects

Default the instruction to a safe behavior, then explicitly select supported cases. The reference computes a legality flag and finally converts illegality or misalignment into a trap.

```
if (!legal || (taken && target[1:0] != 0)) trap = 1;
if (trap) begin
    wr = 0; ld = 0; st = 0; taken = 0;
end
```

A faulting instruction must not partially store a word or write a link before halting. Put a younger store after an invalid instruction and check that it never appears in the write stream.

Run `make learning-labs` and inspect `decode_add`, `word_bytes`, `illegal_branch` and `misaligned_load`. These supplement the baseline regression without changing its reported counts.

If a result differs, decode the instruction word by hand before changing arithmetic. A wrong register field or immediate often looks like a broken ALU.

A research experiment for this part

Write this entry before editing. It is a proposed experiment, not a result already obtained by the learner.

Question	Do signed loads preserve the intended byte value?
Prediction	LB and LBU read the same byte but extend it differently.
Experiment	Store 0x80FF007F, load byte 131 with LB and LBU, compare hand values.
Evidence	Program words, expected register values and word_bytes trace.

What to record

Save source revision, tool versions, input words, seed, exact command, expected and observed behavior, raw artifact path and limitation. End with the decision that the observation supports.

Use `docs/EXPERIMENT_LOG.md`. If the result contradicts the prediction, keep the failure and explain the next test. Do not rewrite the prediction afterward to make the experiment appear successful.

Answers and completion check

2A. No. funct3 and funct7 refine the operation, and opcode 0x13 covers immediate arithmetic separately. Legality checks are part of decode.

2B. Target=8. RISC-V branch offsets are relative to the branch instruction's PC.

2C. The byte strobe is 0100. A word access requires effective_address[1:0]=0; halfwords require bit zero=0.

2D. Each earlier instruction finishes its state update at the preceding edge. The next instruction reads the updated register file during its own full cycle.

Completion check

Decode one R instruction and each immediate format by hand. Demonstrate signed/unsigned loads, a backward branch, a jump link and a rejected encoding. Then add pipeline timing in Volume 3.

Sources and next steps

These sources support the concepts in this volume. The current RTL and local execution contract remain the authority for implementation-specific behavior.

1. RISC-V ISA
2. Project source

Detailed chapter-mapped reading is in `docs/textbook/SOURCES.md`. Research-interest context is in `docs/RESEARCH.md`. The original overview and its interactive figures remain companion material.

Keep the evidence boundary

Teaching labs are separate from the 253-program baseline. Reference coverage and timing numbers belong to the recorded source/tool configuration. No physical board execution, ISA certification or formal proof is implied. The DOT4 speedup covers one kernel.

Continue with Volume 3. Run the complete regression before treating a learning edit as a finished implementation.